

IC526 Advanced Compilers

Lec 07: Parsers

Yoonseung Kim

DGIST EECS

Fall 2026

Updated Course Plan

- Intro
- OCaml
- Rust
- Automata

- Lexer
- Parser
- Type Check
- LLVM IR
- IRGen

PA1 (Warm-Up)

PA2 (Lexer & Parser)

PA3 (Type Checker)

W01 (8/24-26)	W02 (8/31-9/2)	W03 (9/7-9)	W04 (9/14-16)	W05 (9/21-23)	W06 (9/28-30)	W07 (10/7)	Midterm (10/12?)
------------------	-------------------	----------------	------------------	------------------	------------------	---------------	---------------------

- SSA – Mem2Reg
- Optimizations
- Correctness Reasoning

- Backend
- Linker

- Project Presentation

PA4 (IR Generation)

Individual Project (Optimizations / Extensions)

W09 (10/19-21)	W10 (10/26-28)	W11 (11/2-4)	W12 (11/9-11)	W13 (11/16-18)	W14 (11/23-25)	W15 (11/30-12/2)	Final (Report)
-------------------	-------------------	-----------------	------------------	-------------------	-------------------	---------------------	-------------------

Recap: Lexing - ByteStream → TokenStream

- The lexer transforms a **stream of characters** into a **stream of tokens**.
- Today: we transform a **stream of tokens** into an **AST**.

'f' 'n' ' ' 'm' 'a' 'i' 'n' '(' ')' ' ' '{' '\n' '\t' '1' ... eof

↓ Lexing

FN ID("main") () { LET ID("x") = INT(1) + INT(2) ; } EOF

Parsing – TokenStream → AST

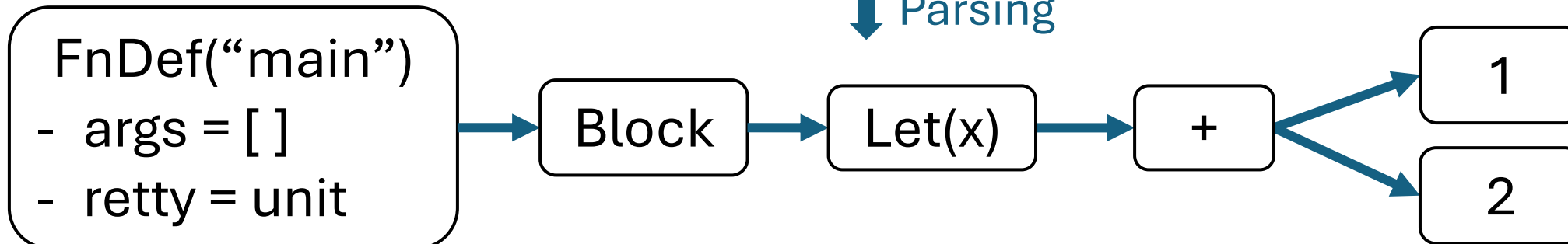
- The lexer transforms a **stream of characters** into a **stream of tokens**.
- Today: we transform a **stream of tokens** into an **AST**.

'f' 'n' ' ' 'm' 'a' 'i' 'n' '(' ')' ' ' ' ' '{' '\n' '\t' '1' ... eof

↓ Lexing

FN ID("main") () { LET ID("x") = INT(1) + INT(2) ; } EOF

↓ Parsing

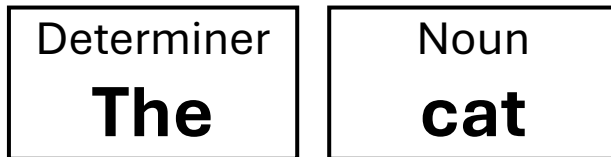


Recap: English Paragraph

Text:

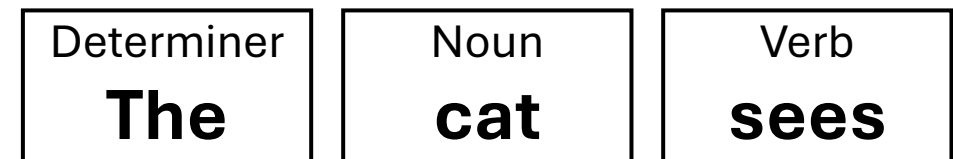
The cat sees a bird. A dog ate my food. She reads old books.

1. Lexical Analysis

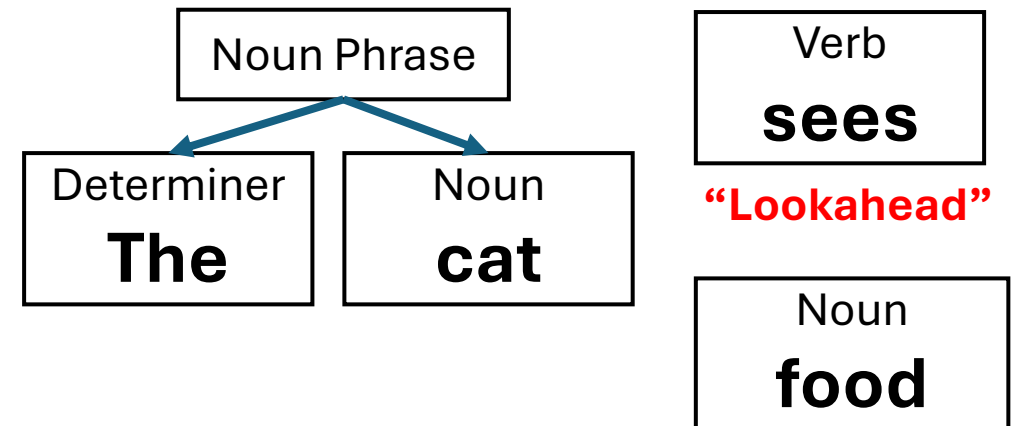


sees a bird. A dog ate my food. She reads old books.

2. Syntax Analysis



a bird. A dog ate my food. She reads old books.



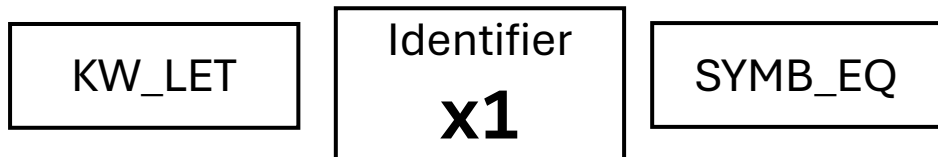
Recap: Reading Program Text

2. Syntax Analysis (by parser)

Text:

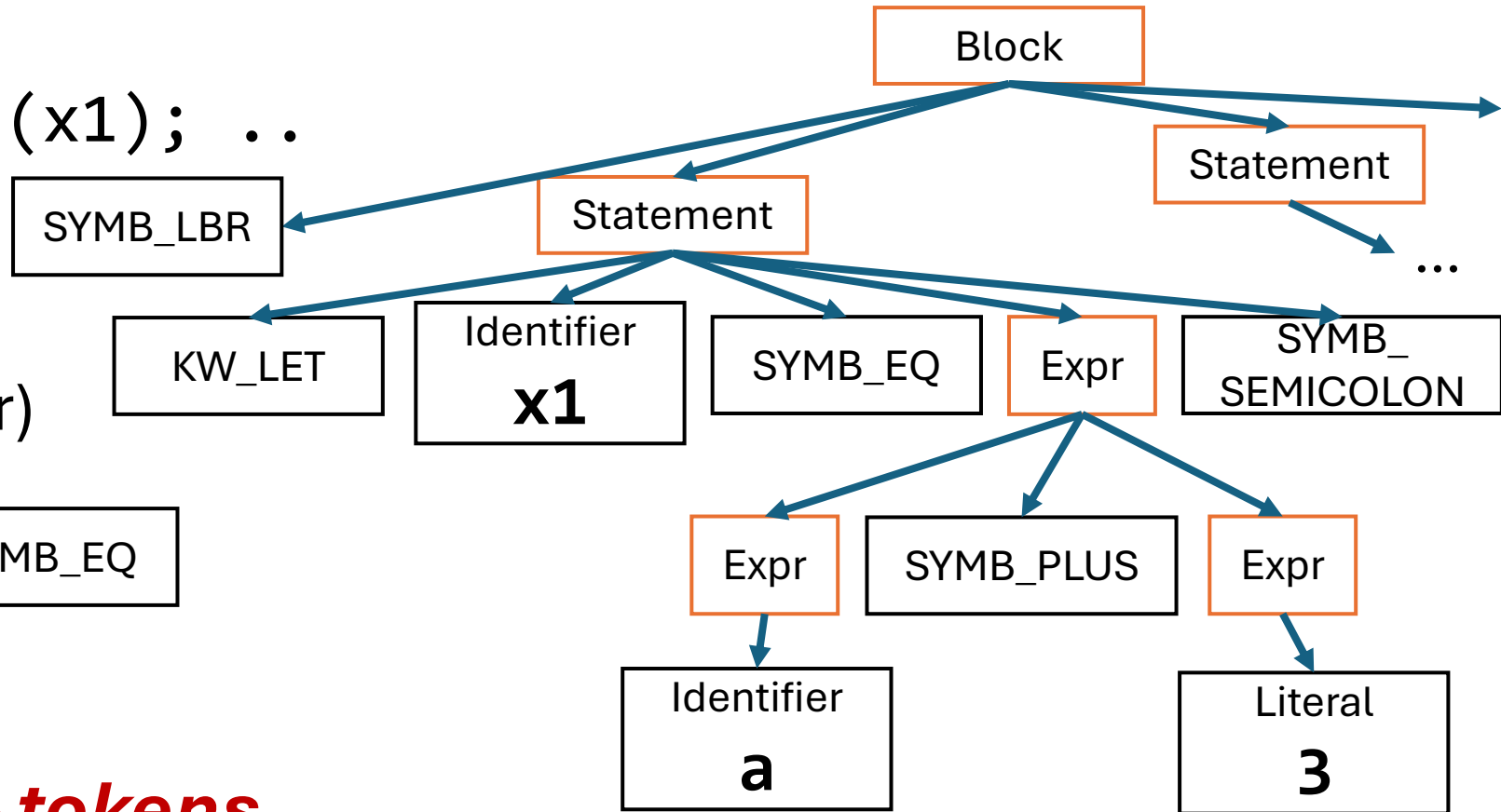
```
{ let x1 = a + 3; f (x1); ..
```

1. Lexical Analysis (by lexer)



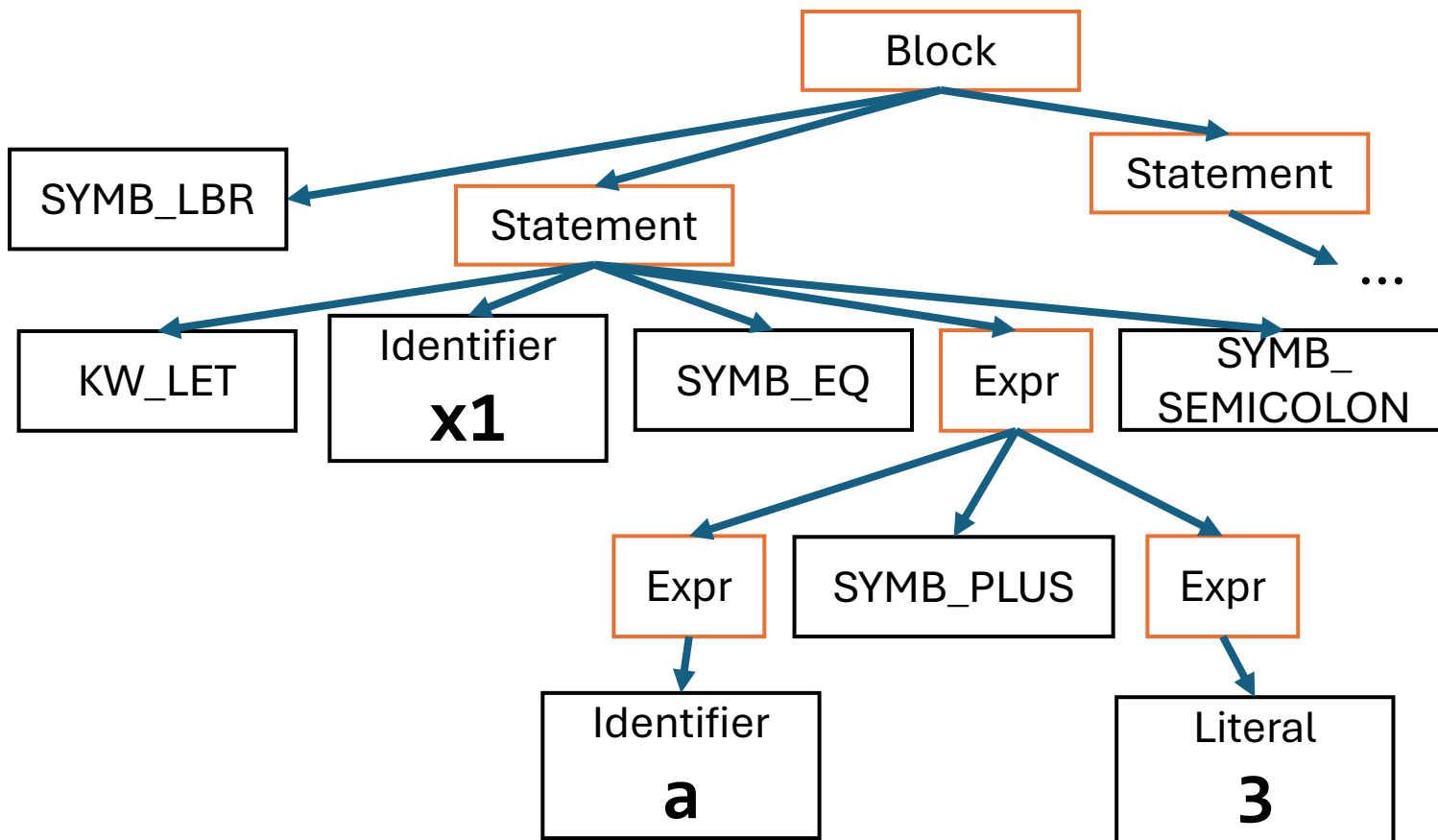
a + 3; f (x1);

From left to right, you recognize ***tokens***.

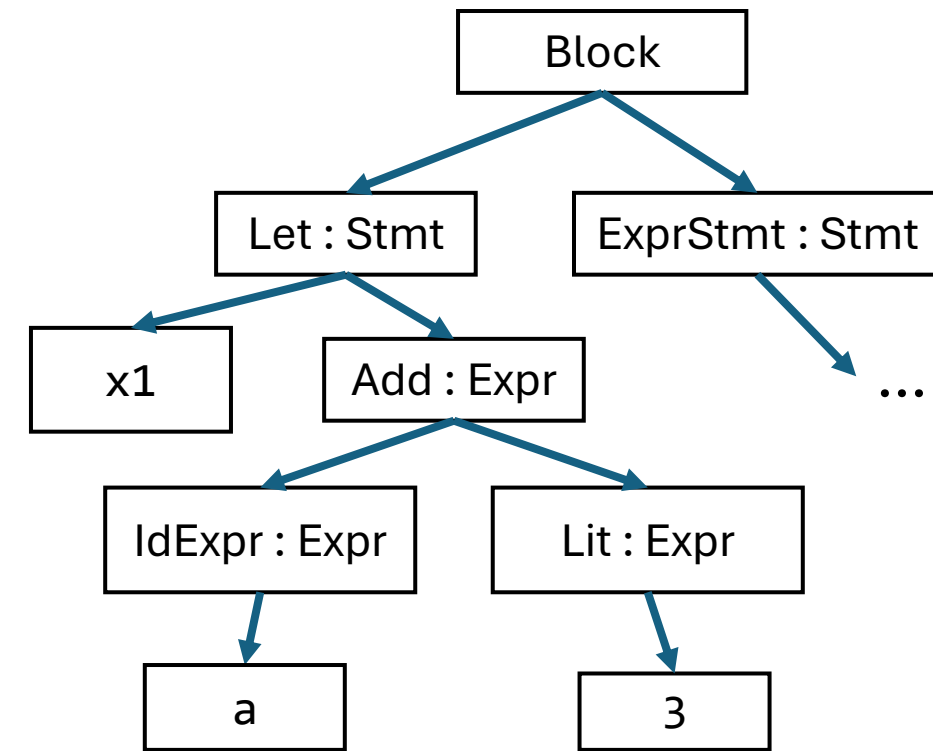


Parser's Role: Construction of AST

The original derivation tree (=parse tree)



Our parser constructs the **AST** directly.

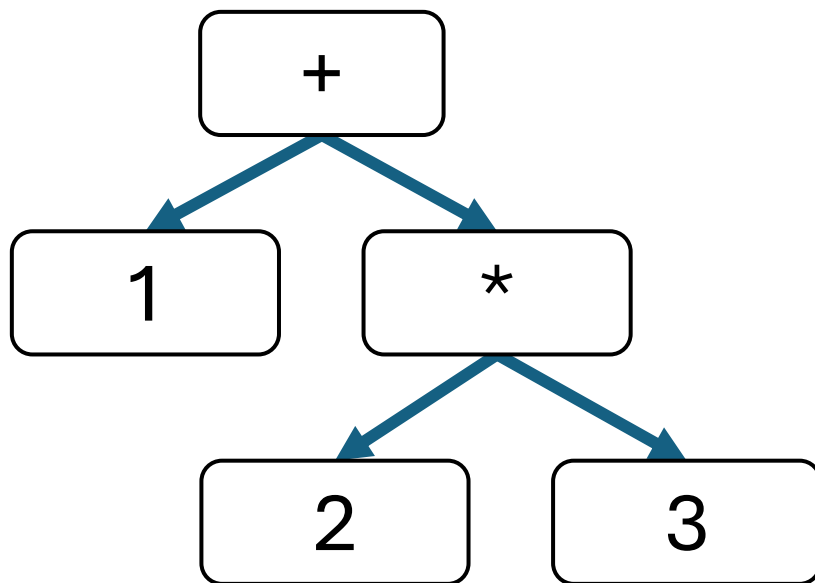


Introduction to Parsers

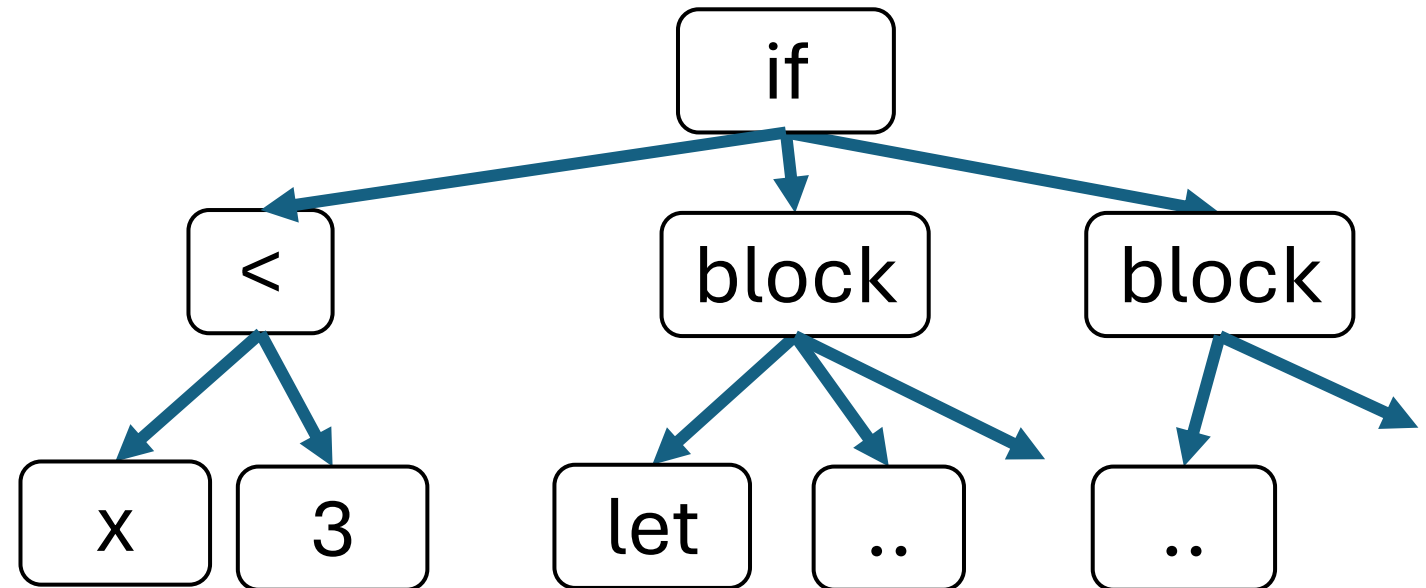
Code Has Tree Structure

- Expressions can be **nested** — and nesting is naturally a tree.
- Program structure can also be represented as a tree.

1 + 2 * 3



if (x < 3) then { let ...; ... } else { ... }



These trees are called ***abstract syntax trees (AST)***.

Why **Abstract** Syntax?

- The AST captures **internal structure**, not surface syntax.

Language 1:

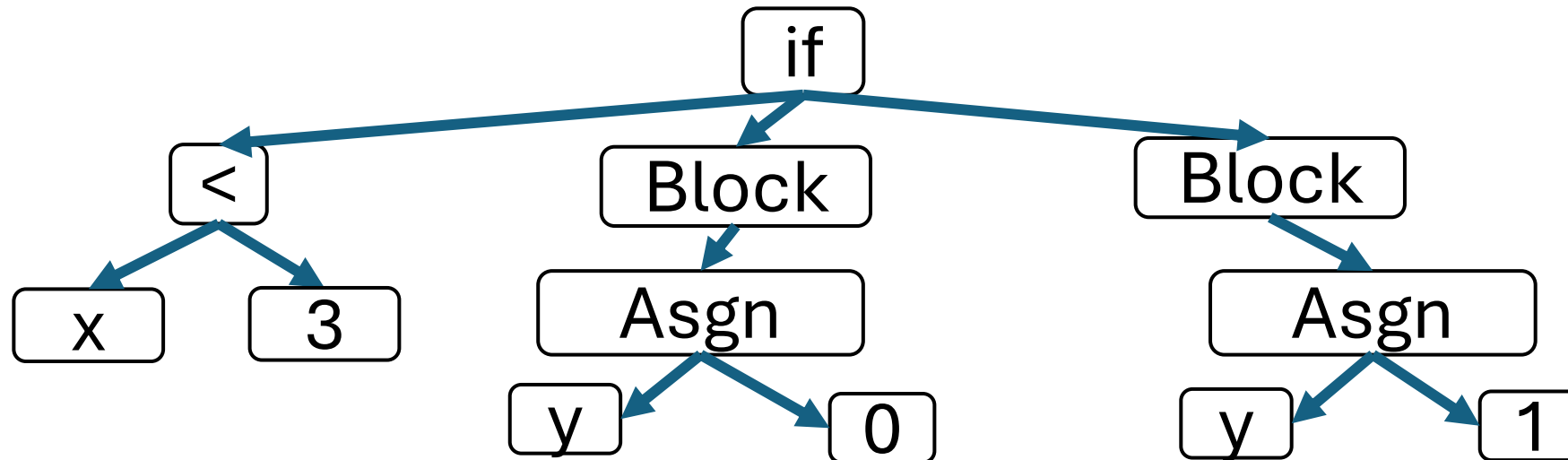
```
if x < 3:  
    y = 0  
else:  
    y = 1
```

Language 2:

```
if (x < 3) {  
    y = 0;  
} else {  
    y = 1;  
}
```

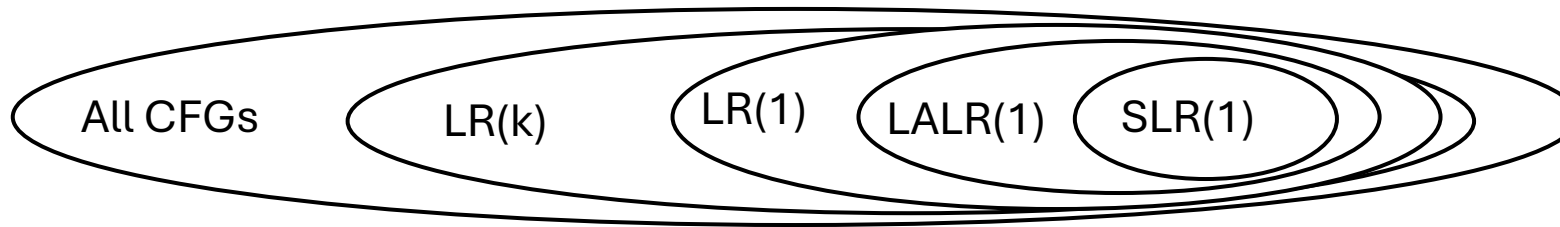
Language 3:

```
if x < 3 then  
    y := 0  
else  
    y := 1  
end
```



Recall: CFG and PDA

- If we describe RustY's syntax using a CFG:
we **may be** able to construct a deterministic parser (DPDA).



- Specifically, if your CFG is an LR(1) grammar,
you can construct an LR(1) parser using a known algorithm.

Our Focus

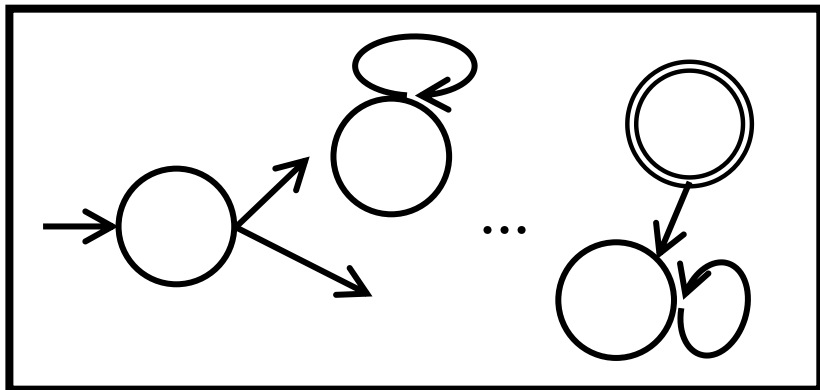
- We will not cover:
 - How to construct parser automata.
 - Comparison between various parser designs: LR(k), LL(k), LALR(k), SLR, ...
- We will study:
 - How LR(1) parsers work.
 - How conflicts arise and how to resolve them.
 - How to get LR(1) parsers using a parser generator tool.

Expectation: You will be able to identify conflicts –
(1) by inspecting a grammar directly, and
(2) by interpreting the conflict reports produced by a parser generator.

How LR(1) Parsers Work

LR(1) Parsers – Widely Used in Practice

- Left-to-right scan, (reverse) rightmost derivation, with 1 token of lookahead.

$$\begin{array}{lcl} S & ::= & E \\ E & ::= & E + E \quad | \quad E * E \\ & | & (E) \quad | \quad n \end{array} \quad (n \in \mathbb{N})$$


Goal:

- Fill in the stack with RHS
- Reduce RHS to LHS
- Until we get S

One lookahead



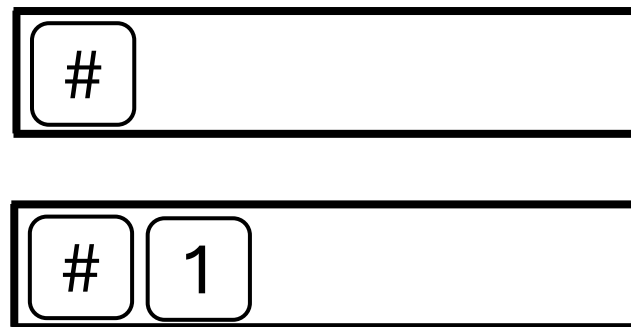
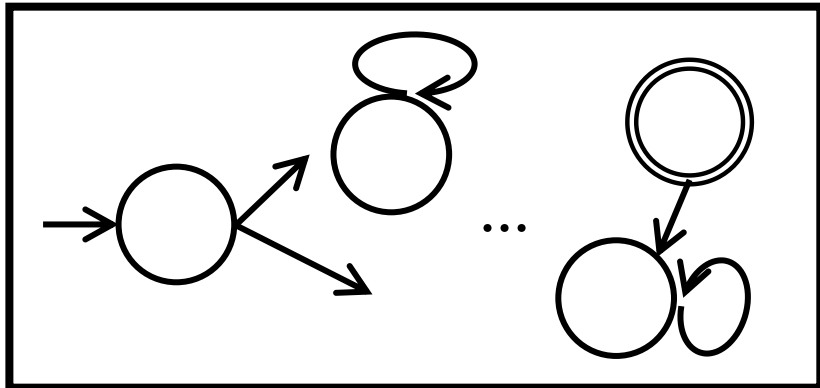
Invisible

LR(1) Parser

- Left-to-right scan, (reverse) rightmost derivation, with 1 token of lookahead.

$$\begin{array}{lcl} S & ::= & E \\ E & ::= & E + E \quad | \quad E * E \\ & | & (E) \quad | \quad n \end{array} \quad (n \in \mathbb{N})$$

(1) The parser may **shift** a token into the stack.

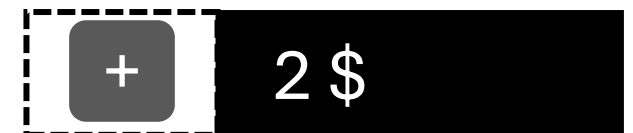
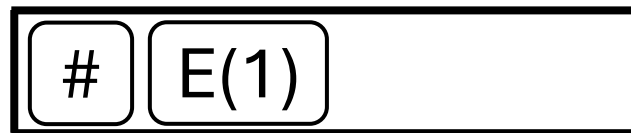
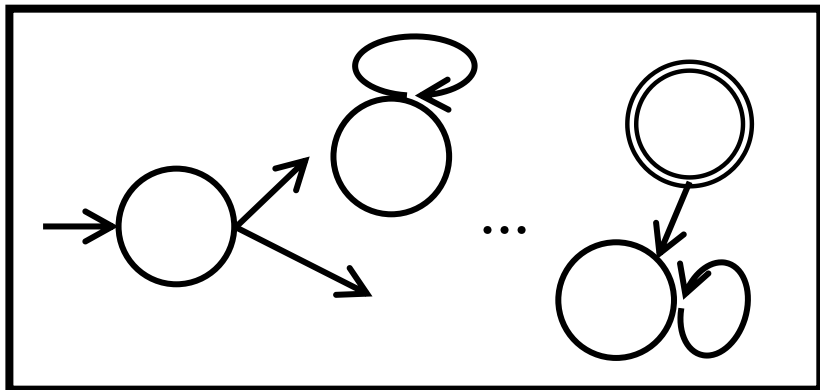


LR(1) Parser

- Left-to-right scan, (reverse) rightmost derivation, with 1 token of lookahead.

$$\begin{array}{lcl} S & ::= & E \\ E & ::= & E + E \quad | \quad E * E \\ & | & (E) \quad | \quad n \quad (n \in \mathbb{N}) \end{array}$$

(2) The parser may **reduce** symbols at the top, according to the rule.



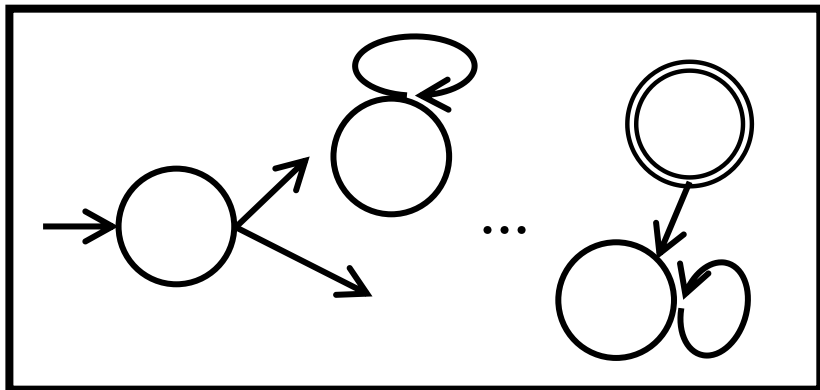
LR(1) Parser

- Left-to-right scan, (reverse) rightmost derivation, with 1 token of lookahead.

$$\begin{array}{lcl} S & ::= & E \\ E & ::= & E + E \quad | \quad E * E \\ & | & (E) \quad | \quad n \end{array} \quad (n \in \mathbb{N})$$

No Shift [+]

There is no way to complete the parse.



#	1
---	---

#	1	+
---	---	---



+	2 \$
---	------

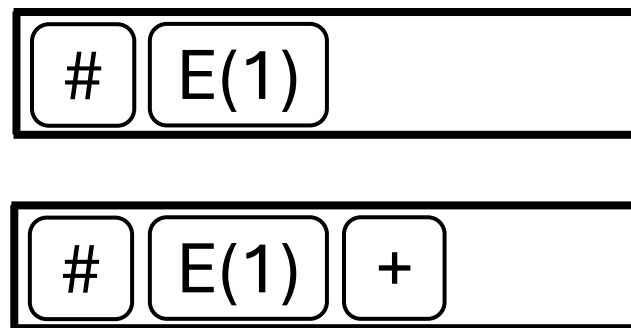
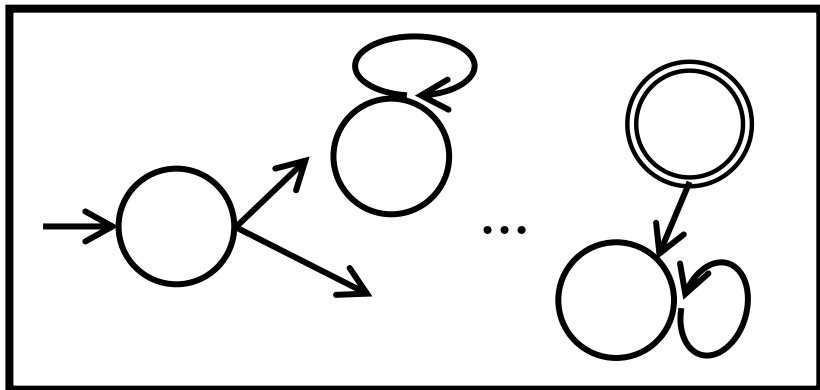
2	\$
---	----

LR(1) Parser

- Left-to-right scan, (reverse) rightmost derivation, with 1 token of lookahead.

$$\begin{array}{lcl} S & ::= & E \\ E & ::= & E + E \quad | \quad E * E \\ & | & (E) \quad | \quad n \end{array} \quad (n \in \mathbb{N})$$

Next move? **Shift [+]**

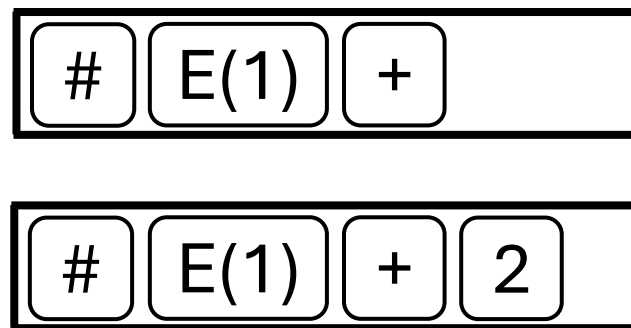
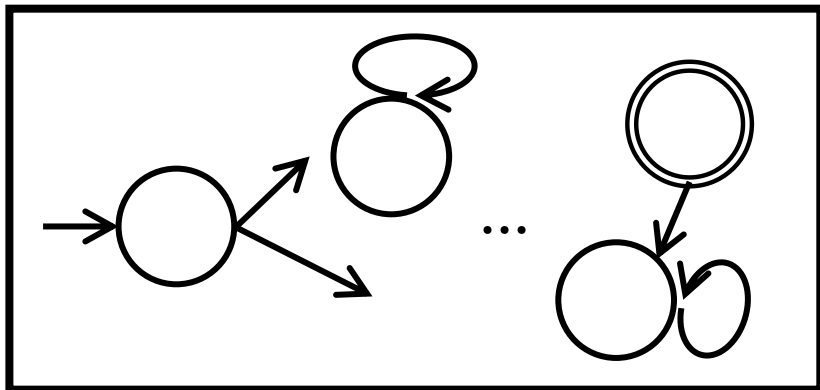


LR(1) Parser

- Left-to-right scan, (reverse) rightmost derivation, with 1 token of lookahead.

$$\begin{array}{lcl} S & ::= & E \\ E & ::= & E + E \quad | \quad E * E \\ & | & (E) \quad | \quad n \end{array} \quad (n \in \mathbb{N})$$

Next move? **Shift [2]**

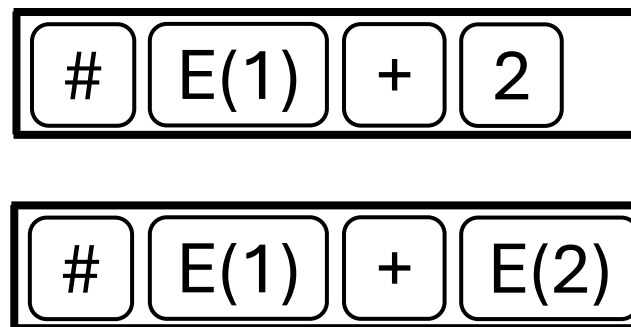
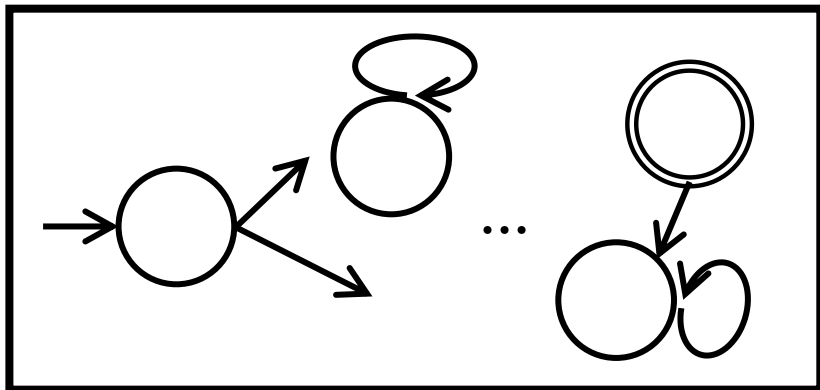


LR(1) Parser

- Left-to-right scan, (reverse) rightmost derivation, with 1 token of lookahead.

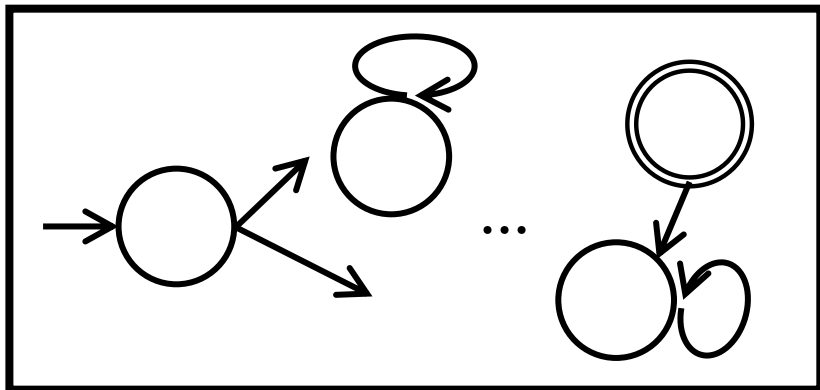
$$\begin{array}{lcl} S & ::= & E \\ E & ::= & E + E \quad | \quad E * E \\ & | & (E) \quad | \quad n \quad (n \in \mathbb{N}) \end{array}$$

Next move? **Reduce [$E \rightarrow 2$]**



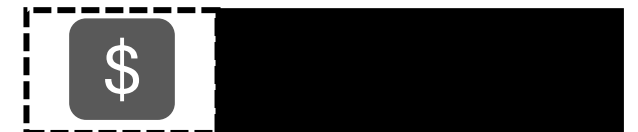
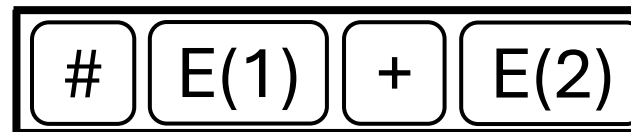
LR(1) Parser

- Left-to-right scan, (reverse) rightmost derivation, with 1 token of lookahead.

$$\begin{array}{lcl} S & ::= & E \\ E & ::= & E + E \quad | \quad E * E \\ & | & (E) \quad | \quad n \end{array} \quad (n \in \mathbb{N})$$


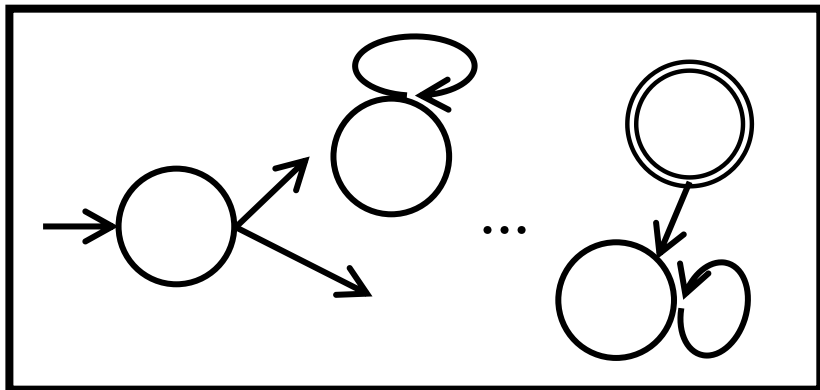
Next move?

Reduce [$E \rightarrow E + E$]

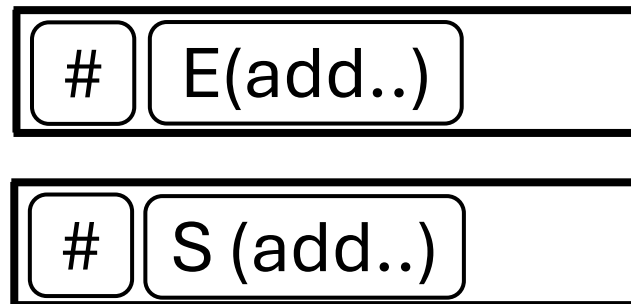


LR(1) Parser

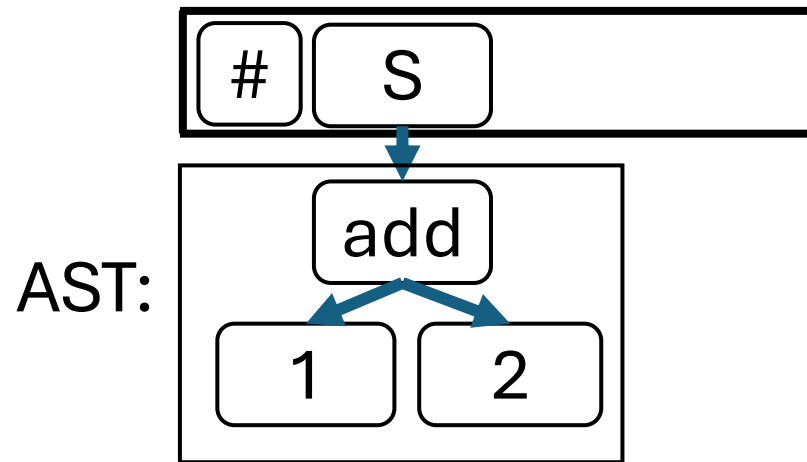
- Left-to-right scan, (reverse) rightmost derivation, with 1 token of lookahead.

$$\begin{array}{lcl} S & ::= & E \\ E & ::= & E + E \quad | \quad E * E \\ & | & (E) \quad | \quad n \end{array} \quad (n \in \mathbb{N})$$


Next move? **Reduce [$S \rightarrow E$]**



LR(1) Parser



- Parsing succeeds when
- all input tokens have been consumed, and
 - reduced to the start symbol S.

Which Grammar is LR(1) and Which Is Not?

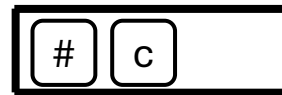
- There is an algorithm for constructing an LR(1) parser.
- If the algorithm successfully constructs an LR(1) parser without **conflicts**, then the grammar is LR(1).
- A grammar may fail to be LR(1) for several reasons:
 - Some grammars require more than one token of lookahead.

$S \rightarrow A a a \mid B a b$

$A \rightarrow c$

$B \rightarrow c$

Why:



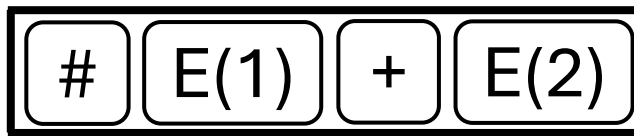
Reduce/reduce conflict!

- Some grammars are **ambiguous**. (In fact, our expression grammar!)
 - ➔ Precedence and associativity declarations can resolve some **shift/reduce conflicts**.

Ambiguity Regarding Precedence

- Recall: a grammar is ambiguous when it allows two different derivation trees for the same input tokens.
- This ambiguity produces a shift/reduce conflict.

Example: 1 + 2 * 3



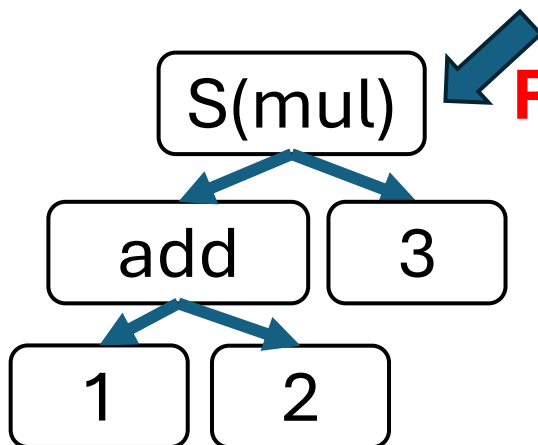
We want to **shift.**

Solution:

Precedence rule

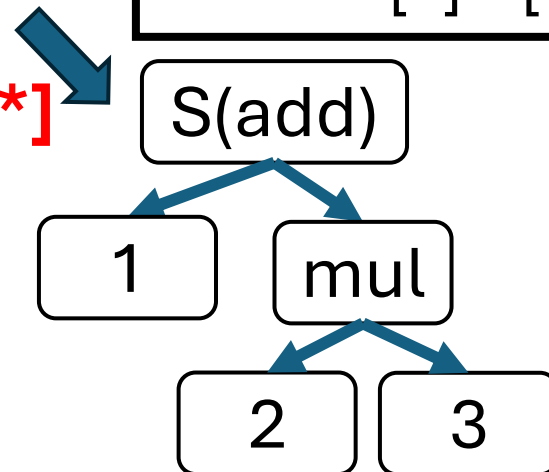
between symbols

$[+] < [*]$



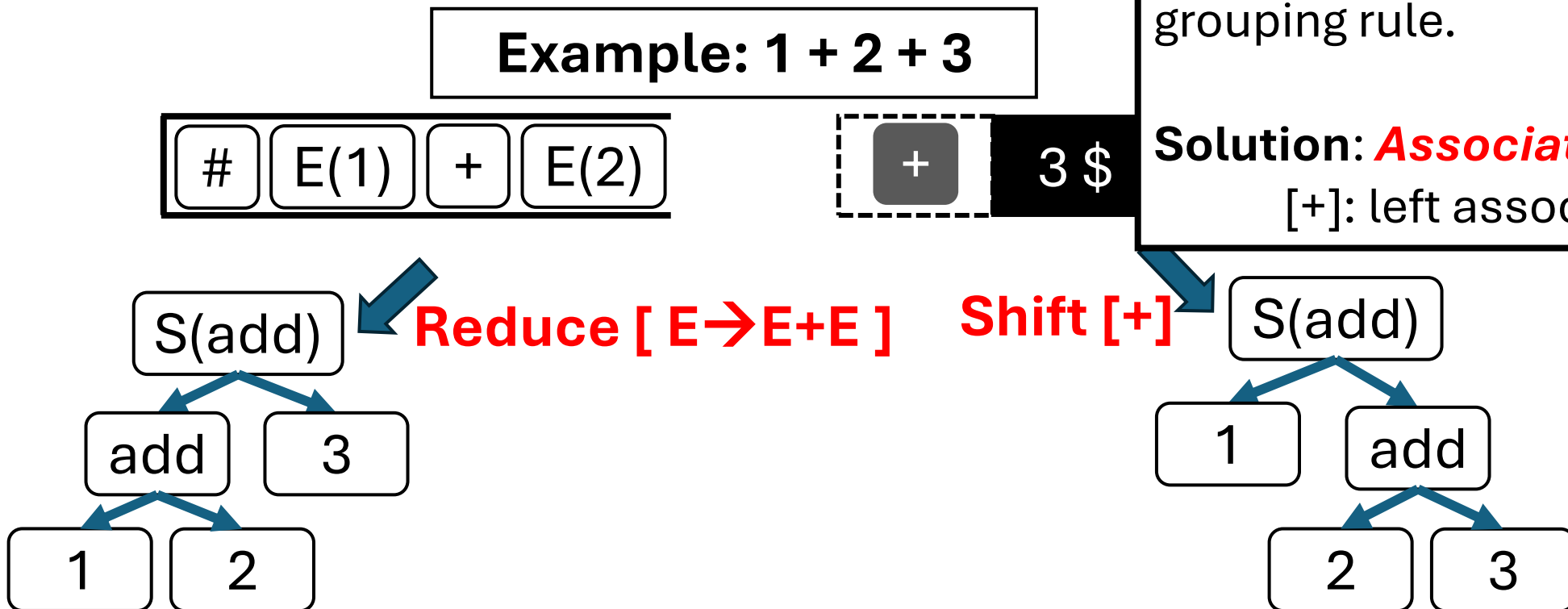
Reduce [$E \rightarrow E + E$]

Shift [*]



Ambiguity Regarding Associativity

- Shift/reduce conflicts happen even with a single symbol.



Addition is associative, but the parser still needs a fixed grouping rule.

Solution: *Associativity rule*
[+]: left associative

Advanced Topic: Stratification

- Alternatively, we can resolve ambiguity by **stratifying** the grammar:

expr	::=	expr + term		term
term	::=	term * factor		factor
factor	::=	INT(n)		(expr)

Precedence: $[*]$ binds more tightly than $[+]$ — $[\text{term}]$ sits below $[\text{expr}]$

Associativity: left-recursive rules $[\text{expr} \rightarrow \text{expr} + \text{term}]$ force left associativity

Same language, different grammar

Why Reverse Rightmost Derivation?

- Rightmost derivation of $1+2*3$:

E

$\rightarrow E + E$

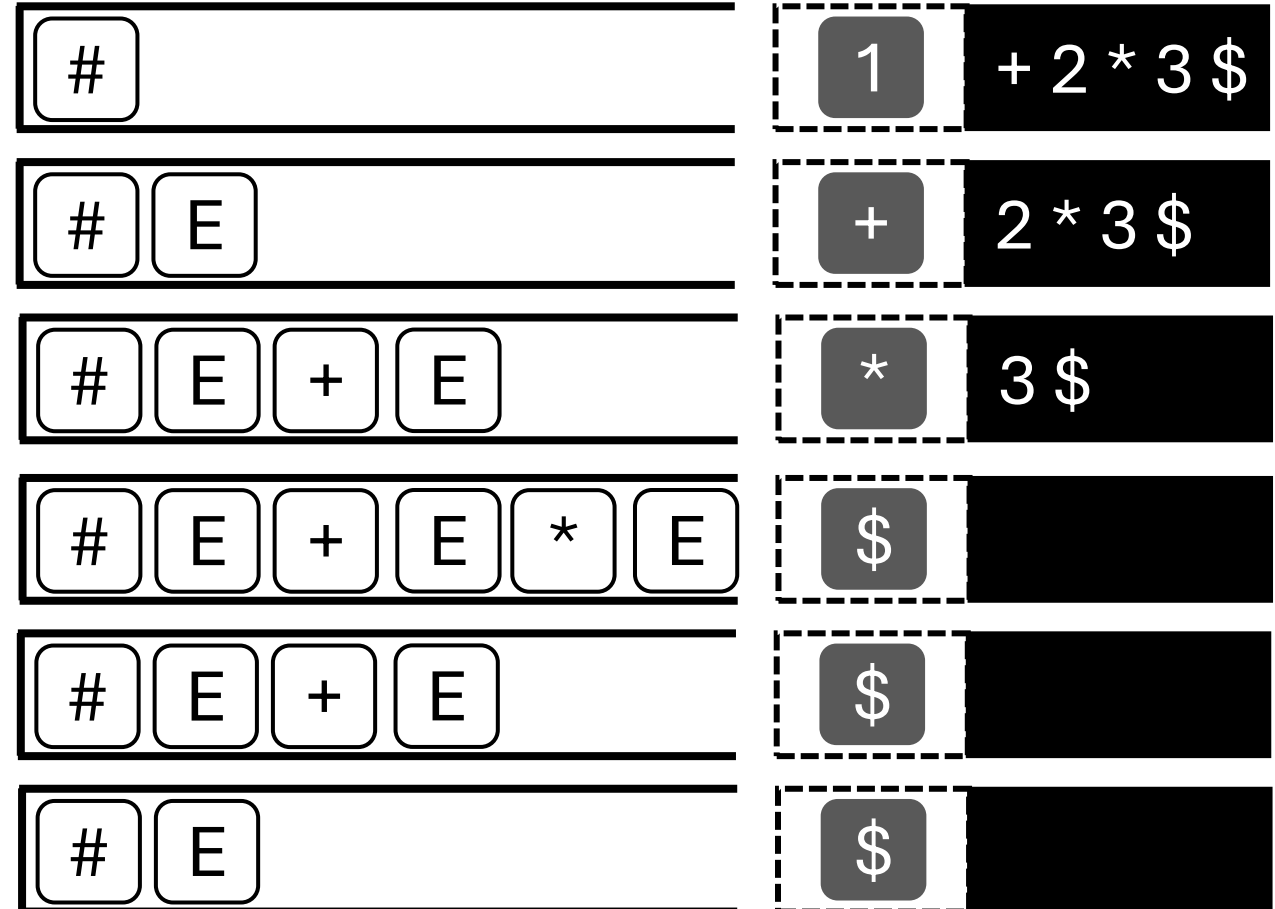
$\rightarrow E + E * E$

$\rightarrow E + E * 3$

$\rightarrow E + 2 * 3$

$\rightarrow 1 + 2 * 3$

- Parsing $1+2*3$



Menhir: A Parser Generator

Menhir: A Parser Generator

- Menhir converts CFG to an LR(1) parser.
- Menhir uses precedence and associativity declarations to resolve conflicts.
- Remaining conflicts are reported; do not rely on their default resolution.



Typical Structure of parser.mly

```
(* 1. ocaml code *)
```

```
%{ open Ast %}
```

```
(* 2. tokens *)
```

```
%token EOF
```

```
%token PLUS MINUS STAR
```

```
%token <int> INT
```

```
%start <expr> main
```

```
%%
```

```
(* 3. CFG *)
```

```
main :
```

```
  | e=exp EOF          { e }
```

```
exp :
```

```
  | l=exp PLUS r=exp    { Add(l,r) }
```

```
  | l=exp MINUS r=exp    { Sub(l,r) }
```

```
  | l=exp STAR r=exp     { Mul(l,r) }
```

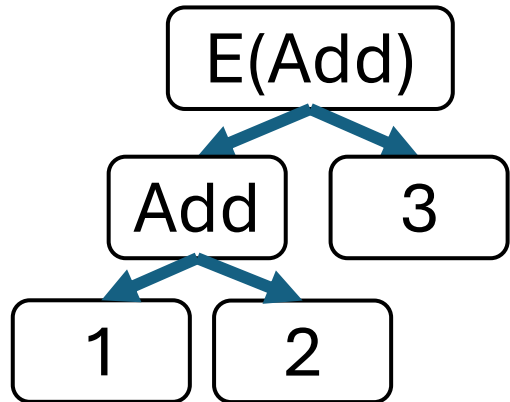
```
  | n=INT                { Num(n) }
```

Format of .mly: 1st Part

%{ open Ast %}

OCaml code – in ast.ml:

```
type expr =  
| Add of (expr * expr)  
| Sub of (expr * expr)  
| Mul of (expr * expr)  
| Num of int
```



=

E[e] where

e : expr =

Add (Add(Num 1, Num 2), Num 3)

Format of .mly: 2nd Part

%token EOF

%token PLUS MINUS STAR

%token <int> INT

} Generates the token type:
(Used by the lexer, too)

```
type token =  
| EOF  
| PLUS  
| MINUS  
| STAR  
| INT of int
```

%left PLUS MINUS

%left STAR

} Associativity & Precedence
STAR > PLUS, MINUS

%start <expr> main

} S = main

- Nonterminal [main] will be defined soon.
- [main] will have a value of type [expr]

%%

Format of .mly: 3rd Part

main :

| e=exp EOF

$E[e] \text{ EOF}$
reduces to
 $\{ e \}$ } $S[e]$

exp :

| l=exp PLUS r=exp

| l=exp MINUS r=exp

| l=exp STAR r=exp

| n=INT

$E[l] + E[r]$
reduces to
 $E[\text{Add}(l, r)]$

$\{ \text{Add}(l, r) \}$
 $\{ \text{Sub}(l, r) \}$
 $\{ \text{Mul}(l, r) \}$
 $\{ \text{Num}(n) \}$ } $\text{INT}[n]$
reduces to
 $E[\text{Num}(n)]$

Menhir Reports Conflicts

%token EOF

%token PLUS MINUS STAR

%token <int> INT

~~%left PLUS MINUS~~

~~%left STAR~~

%start <expr> main

parser.conflicts:

** Conflict (shift/reduce) in state 5.

** Tokens involved: STAR PLUS MINUS

** The following explanations concentrate on token STAR.

** This state is reached from main after reading:

exp STAR exp

...

** In state 5, *looking ahead at STAR, shifting is permitted*

** because of the following sub-derivation:

exp STAR exp

exp . STAR exp

** In state 5, *looking ahead at STAR, reducing* production

** *exp -> exp STAR exp*

** *is permitted* because of the following sub-derivation:

exp STAR exp // lookahead token appears

exp STAR exp .

...

Using the Parser on a String

```
let source = "1 + 2 * 3" in
let lexbuf = Lexing.from_string source in
let ast = Parser.main Lexer.token lexbuf in
ast
(* = Ast.Add (Ast.Num 1,
              Ast.Mul (Ast.Num 2, Ast.Num 3)) *)
```

PA2: Generate Lexer & Parser for RustY

- You will be asked to use ocamllex and Menhir to generate the lexer and parser for your compiler.
- Skeleton code is provided, but you may freely modify it – just keep the provided interfaces unchanged.
- For more information, check out the specification document on the course website.